



Seminario de Programación



Proyecto del ramo e introducción a las metodologías de desarrollo

Patricio Olivares

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Propósito del Proyecto

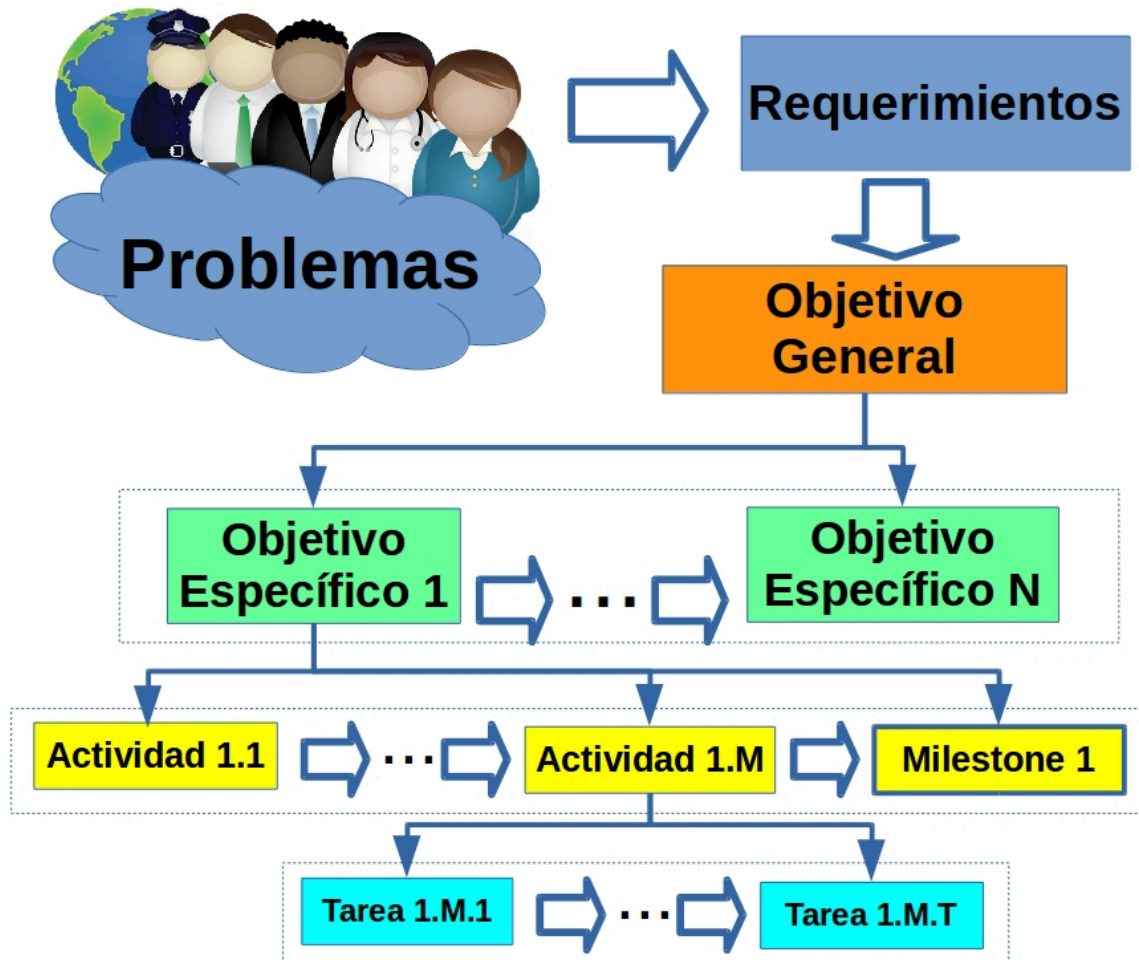
- Desarrollar un **proyecto de software** paso a paso, aplicando los lenguajes **C y C++**.
- Incorporar principios de **metodologías ágiles (Scrum)** y prácticas de **DevOps**.
- Aprender a **colaborar en equipos** utilizando **GitHub** como plataforma principal.
- Preparar una solución con **creciente complejidad**: desde un prototipo en C hasta una aplicación con interfaz en C++.

Objetivos principales

- Hitos:
 - **Hito 0 (finalizado)**: Formación de grupos y definición del tema del proyecto.
 - **Hito 1 (C)**: Implementar un prototipo funcional en C.
 - **Hito 2 (C++ + gráficos)**: Traducir el proyecto a C++ y representar resultados de manera visual.
 - **Hito 3 (C++ + interfaz)**: Integrar una interfaz básica en C++ y consolidar la documentación del proyecto.
- Cada grupo puede **ajustar los hitos** si lo justifica correctamente.
- Se debe registrar el uso de **IA** (herramientas, prompts, ventajas, limitaciones, etc.).

Organización de un Proyecto

- Todo proyecto se organiza en **niveles jerárquicos** que permiten bajar una idea general a tareas concretas:
 - **Objetivo General (OG):** qué se busca lograr en términos globales.
 - **Objetivos Específicos (OE):** pasos concretos y medibles que apoyan al OG.
 - **Actividades:** cómo se alcanzará cada OE.
 - **Tareas:** acciones individuales asignadas a los miembros del equipo.



Objetivo General (Goal)

- El objetivo general es el "**qué**" se espera como resultado del proyecto.
- Es el puente entre los **requerimientos** y la **ejecución** del proyecto.
- Provee una dirección **general** para el equipo (brújula).
- Refleja resultado **deseado**, no necesariamente el real.
- Un proyecto puede tener **varios** objetivos generales

Objetivos Específicos

- Los objetivos específicos son el "**cómo**"
- **Soportan** al objetivo general
- Indican cómo alcanzar (**parte de**) el objetivo general
- Describen un cambio **deseable**, que sea **medible** y alcanzable en el **tiempo disponible**
- Parten con un **verbo de acción**
- Esto permite que **sean medibles**
- Asociados directamente con **hitos** medibles (milestones/productos)
- Deben relacionarse directamente con un **requerimiento**

Construcción de Objetivos Específicos

1. **Cuándo:** Fecha cuando ocurre el cambio.
2. **Quién:** A quién afecta (cuantificar).
3. **Qué:** El cambio deseable, observable y medible que ocurrirá si se cumple.
4. **Cómo:** La manera en la que se efectuar a el cambio.
5. **Criterio:** Cómo se evidenciar a el cambio (métrica)
6. **Nivel de éxito:** Valoraciones del criterio según expectativas

Objetivos - Ejemplos

1. **General:** Los visitantes se convencerán que el calentamiento global existe.
 - **Específicos:**
 - Crear un cuadro comparativo de los costos relacionados con calentamiento global hoy versus 100 años atrás,
 - Ilustrar los efectos del calentamiento global con una galería de fotos.
 - Identificar y explicar los mitos acerca del calentamiento global.
2. **General:** El departamento de seguros médicos incrementará las opciones de proveedores de servicios en un 10%.
 - **Específicos:**
 - Identificar opciones de proveedores y costos.
 - Entrevistar a los consumidores para identificar las opciones que valoran.
 - Comparar con las opciones de los competidores.
3. **General:** Los pacientes esperarán menos de una hora para que un médico los atienda.
 - **Específicos:**
 - Evaluar los requerimientos del personal.
 - Comprar un nuevo SW de organización de citas

Actividades, Hitos (Milestones) y Tareas

- **Actividades**

- Las actividades conforman el **plan de operación** del proyecto.
- Describen **cómo se alcanza** cada objetivo específico de forma detallada y concisa.
- Para cada objetivo, la actividad debiese describir: **quién, qué, cuándo, para qué, cómo, cuántos y el presupuesto.**
- **Hitos**
 - Un **hito** es un **momento específico** dentro de la línea de tiempo del proyecto.
 - Señalan *anclas* para presentar **entregables (resultados) del proyecto**, y requieren seguimiento de avance.
 - En muchos casos, los hitos **no impactan la duración** del proyecto (e.g. MVPs).
 - Se enfocan en **puntos de progreso claves** para lograr el éxito del proyecto.
- **Tareas**
 - Las tareas son el **mínimo nivel** de desagregación en el proyecto.
 - Se refiere al **desglose de tareas atómicas de una actividad** que permiten completarla.
 - Se definen **responsables específicos y plazos.**

Aplicación en los Hitos

- Cada hito debe incluir:
 - **OG y OE claros** documentados en el README.
 - **Actividades y Tareas** gestionadas como *Issues* en GitHub.
 - Entregables definidos según la **Definition of Done (DoD)**.

Definition of Done (DoD)

Un hito se considera finalizado cuando:

- El código compila y se ejecuta correctamente.
- Existen ejemplos de entrada y salida documentados.
- El README está actualizado.
- Las *Issues* asociadas están cerradas en el tablero de GitHub.
- (Opcional) Se genera un **tag de versión**.

De C a C++

Idea central

- No es solo traducir código, es **reorganizar y mejorar el diseño**.

Puntos principales

- Identificar entidades y **agrupar funciones relacionadas en objetos**.
- Separar la lógica en **capas**: dominio, datos, aplicación.

- Mantener carpetas claras: **src, include, docs, tests, data.**
- Documentar en el README cómo evolucionó el código de C a C++.

Hitos

- **Hito 1 (C):** prototipo básico.
- **Hito 2 (C++ + gráficos):** reorganización en clases + visualización.
- **Hito 3 (C++ + interfaz):** interfaz desacoplada de la lógica.

Metodologías Ágiles: Scrum

A continuación veremos **Scrum**, una metodología ágil que utilizaremos como herramienta de organización en el proyecto.

El objetivo es que cada grupo pueda coordinarse mejor, organizar tareas y entregar resultados de forma continua.

¿Qué es Scrum?

- Un marco de trabajo ágil para **gestionar proyectos en ciclos cortos.**
- Se centra en:
 - **Entrega parcial y frecuente de resultados.**
 - **Colaboración constante dentro del equipo.**
 - **Adaptación rápida a cambios y mejoras continuas.**



Elementos principales de Scrum

- **Roles:**
 - *Scrum Master*: facilita la organización y cuida el flujo de trabajo.

- *Product Owner*: mantiene claros los objetivos y entregables.
- *Equipo*: desarrolla el proyecto.
- **Eventos:**
 - *Sprint Planning*: definición de tareas para cada hito.
 - *Daily breve*: reunión corta para ver avances y problemas.
 - *Sprint Review*: presentación de resultados al cierre de cada hito.
 - *Retrospectiva*: reflexión rápida sobre mejoras en la organización.

Scrum en el Proyecto

- **Hitos = Milestones**: cada hito representa un objetivo mayor del proyecto.
- **Sprints dentro de cada hito**: un hito puede organizarse en **varios ciclos Sprint** hasta completar el objetivo.
- **GitHub como soporte**:
 - *Issues y Labels* = tareas organizadas.
 - *Projects (board)* = flujo To-Do → Doing → Done.
 - *Milestones* = los hitos principales del proyecto.
 - *Sprints* = ciclos de trabajo más cortos dentro de cada milestone.

Archivos README e Hitos

- El README debe mostrar el avance del proyecto **por cada hito**:
 - **Al terminar un hito** se debe indicar:
 - Qué se esperaba lograr en ese hito.
 - Cómo se fue alcanzando el objetivo a través de los diferentes Sprints.
 - Qué se espera lograr en el próximo hito.
- De esta manera, el README actúa como una **bitácora de progreso**.

Este es el mínimo de Scrum que se debe aplicar en estos proyectos.

DevOps

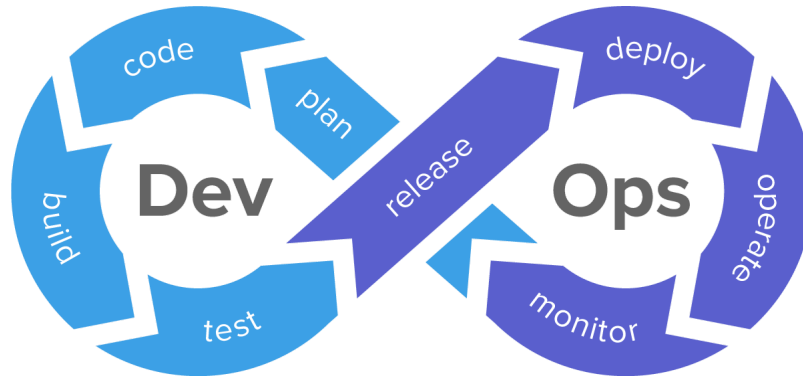
¿Qué es DevOps?

- Conjunto de prácticas que integran:
 - **Desarrollo (Dev)**
 - **Operaciones (Ops)**
 - **Aseguramiento de la calidad (QA)**
- Busca mejorar la colaboración y acelerar la entrega de software.



Ciclo de DevOps

- El trabajo no es lineal, es un **ciclo continuo**:



Cada etapa retroalimenta a la siguiente.

DevOps aplicado al Proyecto

- **GitHub** será nuestra plataforma DevOps:
 - Control de versiones (repositorios, ramas, commits).
 - Gestión del trabajo (Issues, Milestones, Projects).
 - Integración de entregas parciales (cada Sprint/Hito).
- Beneficios:
 - Trabajo más ordenado y transparente.
 - Detección temprana de problemas.
 - Entregas más frecuentes y medibles.

Git y GitHub

¿Por qué Git y GitHub?

-  **Git**: sistema de control de versiones. Permite guardar, comparar y recuperar el historial del proyecto.
-  **GitHub**: plataforma colaborativa. Conecta código, gestión de tareas y trabajo en equipo.
- Uso en el curso: cada grupo gestionará su proyecto completo en GitHub.

Conceptos básicos

- **Repositorio (repo)**: espacio donde vive el proyecto.
- **Commit**: registro de un cambio en el código.
- **Branch (rama)**: línea de desarrollo paralela.
- **Pull Request (PR)**: propuesta de integración de cambios.
- **Issues**: tareas o problemas a resolver.
- **Milestones**: hitos del proyecto (Hito 1, Hito 2, Hito 3).
- **Projects (board)**: tablero de trabajo (To-Do / Doing / Done).

Flujo mínimo de trabajo

1. Crear repositorio en GitHub (con README y .gitignore).
2. Clonar el repo en local.
3. Agregar cambios: `commit`.
4. Subir cambios al remoto: `push`.
5. Coordinar trabajo con ramas y *Pull Requests*.
6. Usar Issues, Milestones y Projects para organizar el trabajo.

Buenas prácticas

- **Commits pequeños y frecuentes**, con mensajes claros.
- **Un Issue por tarea**: fácil de asignar y cerrar.
- **Branches cortos**: más simples de revisar.
- **Pull Requests revisados** por al menos un compañero.

GitHub en el Proyecto

- Cada **hito** será un *Milestone* en GitHub.
- Las **tareas** se gestionan como *Issues*.
- Los **Sprints** se reflejan en el *Project Board*.

- El **README** muestra el avance: lo esperado, lo logrado en los Sprints, y lo próximo por hacer.

Git y GitHub en acción

- Además de la interfaz web y las herramientas de VS Code, usaremos la **consola de Bash** para interactuar con los repositorios.
- La consola permite un **control más preciso** y ayuda a entender cómo funciona Git en el fondo.

Comandos básicos

- **Crear repositorio:** primero en GitHub, luego conectarlo en local con

```
git init
git branch -M main
git remote add origin <url_repo>
```

- **Clonar un repositorio:**

```
git clone <url>
```

- **Actualizar repositorio local:**

```
git pull
```

- **Añadir, remover, renombrar elementos:**

```
git add <file>
git rm <file>
git mv <file>
```

- **Confirmar cambios:**

```
git commit -m "mensaje"
```

- **Enviar cambios al repositorio remoto:**

```
git push -u origin main # (el `-u` se usa solo en el primer push)
```

- **Revisar historial:**

```
git log -p <file>
```

Ejemplo de Proyecto

Tema: Calculadora de promedios de notas

Hitos definidos:

- **Hito 0:** grupo conformado y tema elegido.
- **Hito 1 (C):** crear un programa en C que permita ingresar 3 notas desde la consola y calcule el promedio. Este primer paso es el prototipo funcional mínimo.
- **Hito 2 (C++ + gráficos):** reorganizar el proyecto en C++ creando una clase `Alumno` que almacene notas y calcule el promedio. Generar un gráfico simple con los datos.
- **Hito 3 (C++ + interfaz):** implementar una interfaz básica que permita ingresar varios alumnos, mostrar promedios y documentar la solución.

Organización del Proyecto

- **Objetivo general:** Facilitar el proceso de cálculo y análisis de métricas de rendimiento estudiantil.

Objetivos específicos (por hito)

- **Hito 1:** Implementar un prototipo inicial que permita calcular métricas simples de rendimiento (ej. promedio de notas).
- **Hito 2:** Ampliar el sistema para organizar los datos en estructuras más claras y representar los resultados mediante gráficos básicos.
- **Hito 3:** Incorporar una interfaz sencilla que facilite la interacción del usuario y documentar el proceso de desarrollo y los resultados alcanzados.

Actividades y tareas

- Descomponer cada objetivo específico en actividades concretas (ej. ingresar datos, calcular promedio, generar visualización, diseñar flujo de usuario).
- Registrar cada actividad como una *Issue* en GitHub, asignando responsables.
- Gestionar las tareas en el tablero de proyecto de GitHub (*To-Do* / *Doing* / *Done*), actualizando el estado según el avance.

Scrum aplicado al proyecto

- **Milestones = Hitos:** cada hito se gestiona como un milestone en GitHub.
- **Sprints dentro de cada hito:** en vez de intentar resolver todo de una vez, cada hito se divide en ciclos más pequeños:
 - Ejemplo Hito 2: Sprint 1 (diseño de clase `Alumno`), Sprint 2 (definir gráficos)
- **Roles mínimos:**
 - *Scrum Master:* facilita organización y revisa tableros.
 - *Product Owner:* asegura que los objetivos de cada hito estén claros.
 - *Equipo:* todos los integrantes programan y documentan.

- **Eventos:**
 - *Planning*: definir qué Issues se resuelven en el Sprint.
 - *Daily breve*: 5 minutos para comentar avances y problemas.
 - *Review*: mostrar lo desarrollado en cada hito.
 - *Retrospectiva*: discutir mejoras en la forma de trabajar.
- **Documentación en README**: al finalizar un hito, el grupo escribe qué se esperaba lograr, cómo lo fueron logrando a través de los sprints y qué sigue en el próximo hito.

DevOps aplicado al proyecto

- **Repositorio en GitHub**: cada grupo tendrá un repositorio con:
 - Rama `main` estable (versión siempre lista).
 - Ramas cortas por tarea (`feat/calculo` , `feat/menu`) que luego se integran con *Pull Requests*.
- **Gestión en GitHub**:
 - *Issues* con labels (`feature` , `bug` , `doc`) para organizar el trabajo.
 - *Projects (board)* con columnas To-Do / Doing / Done.
 - *Milestones* para agrupar Issues de cada hito.
- **Ciclo de entregas frecuentes**: cada Sprint termina con un commit estable o un pequeño tag (`v0.1-h1` , `v0.2-h2`).
- **Definition of Done (DoD)**: un entregable está completo cuando:
 - El código compila y corre sin errores.
 - Hay ejemplos de entrada/salida documentados.
 - El README está actualizado con el progreso.
 - Las Issues correspondientes están cerradas.

Resumen

Este ejemplo muestra cómo un proyecto muy simple, una calculadora de promedios, permite:

- Practicar la **traducción de C a C++** y la evolución de un código simple a uno más organizado.
- Aplicar **Scrum** como marco de organización con sprints, roles y reuniones básicas.
- Integrar prácticas de **DevOps**, usando GitHub para versionar, organizar y documentar avances.
- Mantener la trazabilidad del proyecto a través de Issues, tableros, milestones y actualizaciones en el README.

Actividad práctica: Primer proyecto en GitHub

Objetivo: Crear un repositorio en GitHub, asociarlo a un proyecto (board) y subir un código inicial.

Pasos a seguir

1. Crear el repositorio en GitHub

- Nombre sugerido: `proyecto-tel102-grupoX`
- Incluir `README.md` y `.gitignore` (opción C/C++).
- Mantener la rama principal como `main`.

2. Clonar el repositorio

- Desde VS Code o consola:

```
git clone <url>  
cd proyecto-tel102-grupoX
```

3. Agregar un archivo inicial

- Crear la carpeta `src/` y dentro un archivo `hello.c` con el siguiente contenido:

```
#include <stdio.h>  
int main() {  
    printf("Hola TEL102!\n");  
    return 0;  
}
```

4. Hacer commit y push

- En consola o VS Code:

```
git add .  
git commit -m "Primer commit: hola mundo"  
git push -u origin main
```

(Si no has configurado tu llave SSH, sigue las siguientes [instrucciones](#))

5. Crear un Project en GitHub

- Ir a la pestaña **Projects** del repositorio.
- Crear un tablero con 3 columnas: *To-Do*, *Doing*, *Done*.
- Agregar al menos **un Issue** (ej. "Subir hola mundo") y moverlo en el tablero.

Resultado esperado

- Repositorio creado con `README.md` y código inicial en `src/hello.c`.
- Primer commit reflejado en GitHub.
- Tablero de proyecto configurado con al menos 1 Issue gestionado.